

Practical Week-5

Task 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
lenovo@Manya:~$ mkdir ProducerConsumer
lenovo@Manya:~$ cd ProducerConsumer
lenovo@Manya:~/ProducerConsumer$ pwd
/home/lenovo/ProducerConsumer
lenovo@Manya:~/ProducerConsumer$ nano producer_consumer.c
lenovo@Manya:~/ProducerConsumer$ gcc producer_consumer.c -o producer_consumer
lenovo@Manya:~/ProducerConsumer$ ls
producer_consumer  producer_consumer.c
lenovo@Manya:~/ProducerConsumer$ ./producer_consumer
Consumer: Received 400000 bytes

Producer: Sent 400000 bytes
Communication time: 0.000787 seconds
Data transferred: 0.38 MB
Communication throughput: 484.76 MB/s
lenovo@Manya:~/ProducerConsumer$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define DATA_SIZE 100000

int main()
{
    int pipefd[2];
    pid_t pid;

    int data[DATA_SIZE];

    struct timespec start, end;

    // Create anonymous pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Create child process
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    // Parent process - Producer
    if (pid > 0)
    {
        close(pipefd[0]); // Close reading end
```

```

// Generate data
for (int i = 0; i < DATA_SIZE; i++)
{
    data[i] = i + 1;
}

// Start timing
clock_gettime(CLOCK_MONOTONIC, &start);

// Send data through pipe
ssize_t total_written = 0;
char *ptr = (char *)data;
ssize_t remaining = sizeof(data);

while (remaining > 0)
{
    ssize_t n = write(pipefd[1], ptr, remaining);

    if (n <= 0)
    {
        perror("write");
        exit(EXIT_FAILURE);
    }

    total_written += n;
    ptr += n;
    remaining -= n;
}

close(pipefd[1]); // Close writing end

wait(NULL);      // Wait for child

// End timing
clock_gettime(CLOCK_MONOTONIC, &end);

double time_taken =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1e9;

```

```
lenovo@Manya
GNU nano 8.7.1
double data_mb = (double)total_written / (1024 * 1024);
double throughput = data_mb / time_taken;

printf("\nProducer: Sent %ld bytes\n", total_written);
printf("Communication time: %.6f seconds\n", time_taken);
printf("Data transferred: %.2f MB\n", data_mb);
printf("Communication throughput: %.2f MB/s\n", throughput);
}

// Child process - Consumer
else
{
    close(pipefd[1]); // Close writing end

    int received;
    ssize_t total_read = 0;

    char buffer[4096];

    // Read data from pipe
    while (1)
    {
        ssize_t n = read(pipefd[0], buffer, sizeof(buffer));

        if (n < 0)
        {
            perror("read");
            exit(EXIT_FAILURE);
        }

        if (n == 0)
            break;

        total_read += n;
    }

    close(pipefd[0]);

    printf("Consumer: Received %ld bytes\n", total_read);
}

return 0;
}
```

Task2: Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```

lenovo@Manya:~/ProducerConsumer$ ls -l | grep ".c"
-rwxr-xr-x 1 lenovo      lenovo      16400 Sep 17 13:31 producer_consumer
-rw-r--r-- 1 lenovo      lenovo      2525 Sep 17 13:31 producer_consumer.c
lenovo@Manya:~/ProducerConsumer$ cd ..
lenovo@Manya:~$ mkdir ShellPipe
lenovo@Manya:~$ cd ShellPipe
lenovo@Manya:~/ShellPipe$ nano ls_grep.c
lenovo@Manya:~/ShellPipe$ gcc ls_grep.c -o ls_grep
lenovo@Manya:~/ShellPipe$ ls
ls_grep  ls_grep.c
lenovo@Manya:~/ShellPipe$ ./ls_grep
-rw-r--r-- 1 lenovo      lenovo      1468 Sep 17 13:33 ls_grep.c
lenovo@Manya:~/ShellPipe$ ls -l | grep ".c"
-rw-r--r-- 1 lenovo      lenovo      1468 Sep 17 13:33 ls_grep.c
lenovo@Manya:~/ShellPipe$ |

```

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Create first child for ls
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid1 == 0)
    {
        // Child 1: ls -l

        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        // Close unused pipe ends
        close(pipefd[0]);
        close(pipefd[1]);
    }
}

```

```
lenovo@Manya
GNU nano 8.7.1

    // Execute ls -l
    execlp("ls", "ls", "-l", (char *)NULL);

    // If execlp fails
    perror("execlp ls");
    exit(EXIT_FAILURE);
}

// Create second child for grep
pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    exit(EXIT_FAILURE);
}

if (pid2 == 0)
{
    // Child 2: grep ".c"

    // Redirect stdin from pipe
    dup2(pipefd[0], STDIN_FILENO);

    // Close unused pipe ends
    close(pipefd[1]);
    close(pipefd[0]);

    // Execute grep ".c"
    execlp("grep", "grep", ".c", (char *)NULL);

    // If execlp fails
    perror("execlp grep");
    exit(EXIT_FAILURE);
}
```

```
    // If execlp fails
    perror("execlp grep");
    exit(EXIT_FAILURE);
}

// Parent process
close(pipefd[0]);
close(pipefd[1]);

// Wait for both children
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

return 0;
}
```